VisionPong

A Computer Vision based approach for motion controlled video games



By: Aneesh Pabolu
TJHSST Computer Systems Lab, Dr. Yilmaz

BACKGROUND

What are motion-controlled games?

Simply put, motion-controlled games are games in which the main character or avatar in the game moves based on real physical movements a human exerts.

Currently 2 main types of motion-controlled games

Sensor based (a sensor ‘sees’ your movement, usually IR): 	Controller based (a motion tracking controller ‘feels’ your movement by moving with you): 

Motion controlled games are video games that make physical movement rewarding and fun!

For context, here is how Xbox Kinect works:



Problem

- Motion-controlled games involve special and expensive hardware (does not include cost of games):
  - Xbox Kinect + Xbox 360: $299.99 (minimum)
  - Xbox Kinect + Xbox One: $499.99 (minimum)
  - Nintendo Switch (controller based): $299.99 (minimum)
  - Wii (controller based): $249.99 (minimum)





- Most motion-controlled games today involve physical motion controllers (Nintendo Switch Joy-cons for example)

- Most Sensor based motion-controlled games are obsolete! (This is what I want to solve!)

OTHER SOLUTIONS

Only modern solution is the Nex Playground.

Cost: $249 for the external console

Main issue: Cost & Limited set of games and require the actual device to play those games



Fig. 1

Fig. 2



My solution

In simple terms, my solution uses only your webcam and your computer's processing power to detect your movement to play motion-controlled games.





Your computer and webcam!

Let’s Get a better understanding of my solution…

How Xbox Kinect work:



Source 1

(Expected) Overview:



Novelty & Solution



Fig. 3

Solution & Goal:

- To create a motion-controlled Table Tennis simulation to explore how to build a framework for future motion-controlled simulations based on Computer Vision

Novelty:

- Camera and Computer Vision based motion-tracking game
- Require no external hardware other than your laptop/computer

Impact

- Physical activity and health:
  - Non-sedentary indoor fun!
  - Games like Table Tennis, Bowling, etc. will require you to move your arms, legs and body: indoor exercise!
- More Accessibility in motion-controlled gaming:
  - Removing external hardware requirements

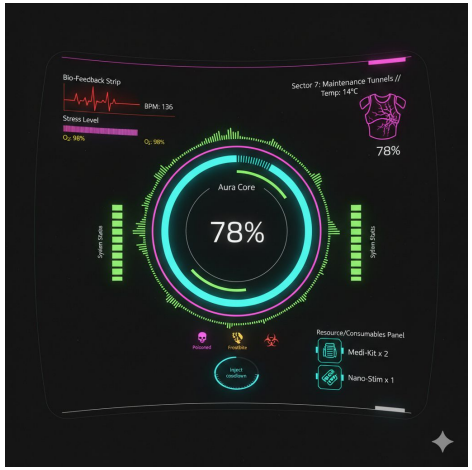
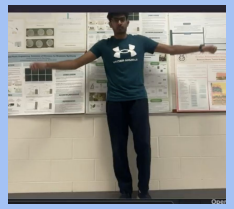


Fig. 4

Method Breakdown (3 major steps):



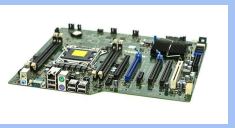




```
Coordinates = {
   Head: (0, 5, 2)
   Left_Arm: (5, 4, 2)
   Right_Arm: (6, 3, 2)
   Left_Knee: (4, 7, 2)
   Right_Knee: (6, 7, 2)
   .
   .
}
```

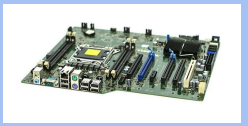
Step 1: Extracting coordinates

```
Coordinates = {
   Head: (0, 5, 2)
   Left_Arm: (5, 4, 2)
   Right_Arm: (6, 3, 2)
   Left_Knee: (4, 7, 2)
   Right_Knee: (6, 7, 2)
   .
   .
}
```



Host, Port 5052

Step 2: Sending Data to the computer locally

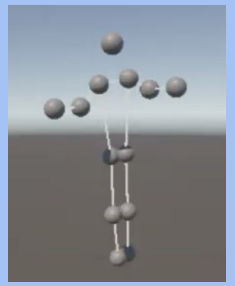


Host, Port 5052

```
Coordinates = {
   Head: (0, 5, 2)
   Left_Arm: (5, 4, 2)
   Right_Arm: (6, 3, 2)
   Left_Knee: (4, 7, 2)
   Right_Knee: (6, 7, 2)
   .
   .
}
```

C#
Scripts

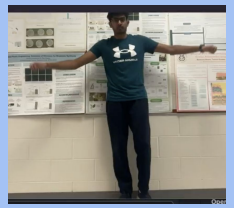
+ unity



Step 3: Unity reads coordinates from the local computer and mirrors the digital avatar

STEP 1 BREAKDOWN...







```
Coordinates = {
   Head: (0, 5, 2)
   Left_Arm: (5, 4, 2)
   Right_Arm: (6, 3, 2)
   Left_Knee: (4, 7, 2)
   Right_Knee: (6, 7, 2)
   .
   .
}
```

Step 1: Extracting coordinates

```
Coordinates = {
   Head: (0, 5, 2)
   Left_Arm: (5, 4, 2)
   Right_Arm: (6, 3, 2)
   Left_Knee: (4, 7, 2)
   Right_Knee: (6, 7, 2)
   .
   .
}
```



Host, Port 5052

Step 2: Sending Data to the computer locally

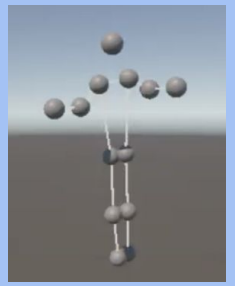


Host, Port 5052

```
Coordinates = {
   Head: (0, 5, 2)
   Left_Arm: (5, 4, 2)
   Right_Arm: (6, 3, 2)
   Left_Knee: (4, 7, 2)
   Right_Knee: (6, 7, 2)
   .
   .
}
```

C#
Scripts

+ unity



Step 3: Unity reads coordinates from the local computer and mirrors the digital avatar

STEP 1: Extracting coordinates

What is MediaPipe?

Mediapipe is an open source ML model that can take an image and identify to coordinates of important body features each called a landmark:

- Eyes
- Arms & fingers
- Feet & Legs

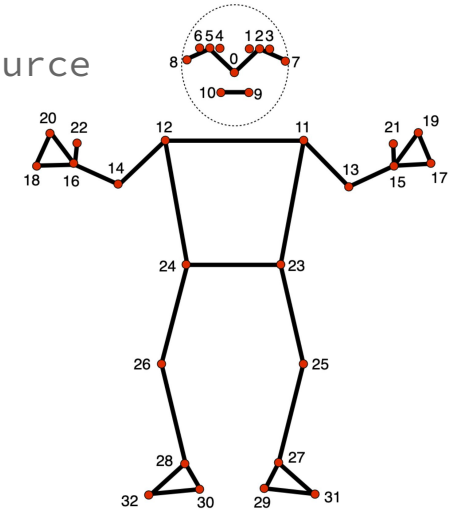


Fig. 7

```
Coordinates = {
   Head: (0, 5, 2)
   Left_Arm: (5, 4, 2)
   Right_Arm: (6, 3, 2)
   ...
}
```

outputs coordinates

Main Loop for Tracking

```python
pose_model = mp.solutions.pose # MediaPipe's Pose Estimation model

with pose_model.Pose(
    # model parameters
) as pose:

    while True:
        ret, frame = cap.read()
        if not ret:
            break

        rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
        results = pose.process(rgb)

        data = results.pose_landmarks

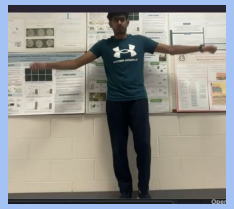
        if data:
            # Adding movement based on the Nose (landmark 0)
            nose = data.landmark[0]
            # Convert to centered coordinates
            xNose = nose.x - 0.5
            yNose = 0.5 - nose.y
            zNose = -nose.z

            # Adding movement based on the Right Wrist (landmark 16)
            wrist_r = data.landmark[16]
            # Convert to centered coordinates
            xWrist_r = wrist_r.x - 0.5
            yWrist_r = 0.5 - wrist_r.y
            zWrist_r = -wrist_r.z

            # Adding movement based on the Left Wrist
            wrist_l = data.landmark[15]
            xWrist_l = wrist_l.x - 0.5
            yWrist_l = 0.5 - wrist_l.y
            zWrist_l = -wrist_l.z
```

Step 2 Breakdown...



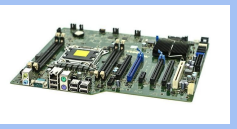




```
Coordinates = {
   Head: (0, 5, 2)
   Left_Arm: (5, 4, 2)
   Right_Arm: (6, 3, 2)
   Left_Knee: (4, 7, 2)
   Right_Knee: (6, 7, 2)
   .
   .
}
```

Step 1: Extracting coordinates

```
Coordinates = {
   Head: (0, 5, 2)
   Left_Arm: (5, 4, 2)
   Right_Arm: (6, 3, 2)
   Left_Knee: (4, 7, 2)
   Right_Knee: (6, 7, 2)
   .
   .
}
```



Host, Port 5052

Step 2: Sending Data to the computer locally

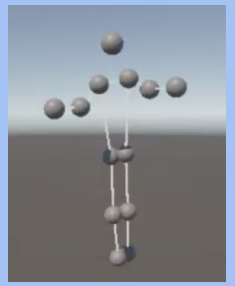


Host, Port 5052

```
Coordinates = {
   Head: (0, 5, 2)
   Left_Arm: (5, 4, 2)
   Right_Arm: (6, 3, 2)
   Left_Knee: (4, 7, 2)
   Right_Knee: (6, 7, 2)
   .
   .
}
```

C#
Scripts

+ unity



Step 3: Unity reads coordinates from the local computer and mirrors the digital avatar

STEP 2.



```
Coordinates = {
   Head: (0, 5, 2)
   Left_Arm: (5, 4, 2)
   Right_Arm: (6, 3, 2)
   ...
}
```

(SENT THROUGH A SOCKET)

Host,
Port

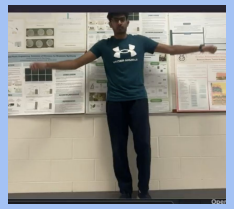
Port: End/start of communication
Socket: Live connection (Analogy: wire)

```
HOST = "localhost"
PORT = 5052
CAM_INDEX = 0
```

```
# UDP socket                    # Use IPv4          # UDP doesn't check if info sent or received
sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
cap = cv2.VideoCapture(CAM_INDEX)
```

Step 3 Breakdown...



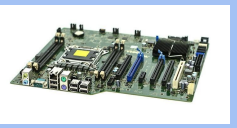




```
Coordinates = {
   Head: (0, 5, 2)
   Left_Arm: (5, 4, 2)
   Right_Arm: (6, 3, 2)
   Left_Knee: (4, 7, 2)
   Right_Knee: (6, 7, 2)
   .
   .
}
```

Step 1: Extracting coordinates

```
Coordinates = {
   Head: (0, 5, 2)
   Left_Arm: (5, 4, 2)
   Right_Arm: (6, 3, 2)
   Left_Knee: (4, 7, 2)
   Right_Knee: (6, 7, 2)
   .
   .
}
```



Host, Port 5052

Step 2: Sending Data to the computer locally

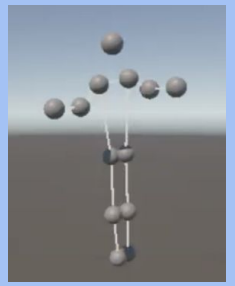


```
Coordinates = {
   Head: (0, 5, 2)
   Left_Arm: (5, 4, 2)
   Right_Arm: (6, 3, 2)
   Left_Knee: (4, 7, 2)
   Right_Knee: (6, 7, 2)
   .
   .
}
```

Host, Port 5052

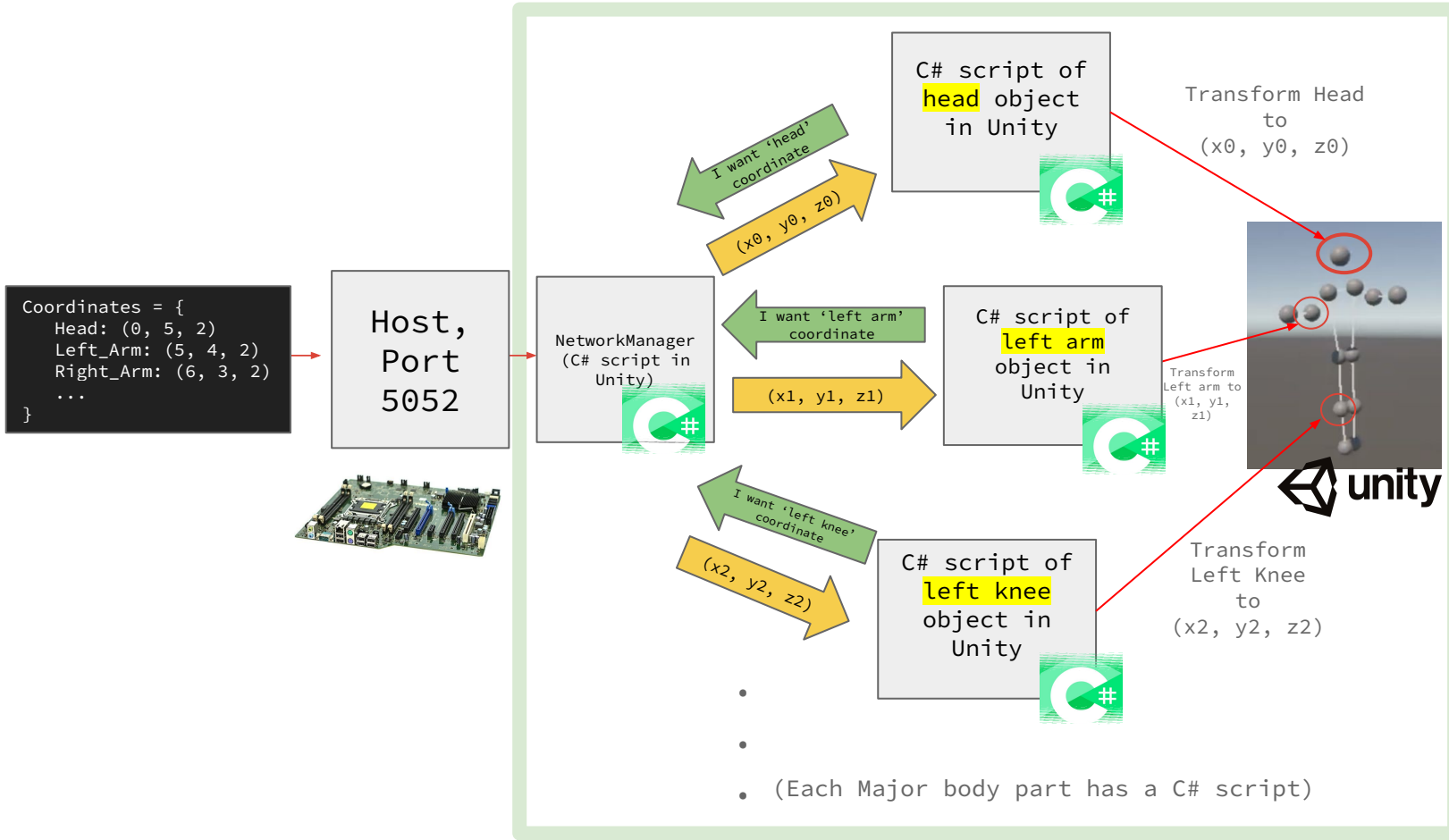
C#
Scripts

+ unity



Step 3: Unity reads coordinates from the local computer and mirrors the digital avatar

Step 3: (Now we know the coordinate of each part of the body. How do we make our avatar move accordingly?)



C# Scripts

```
Data {
   Head: (x0, y0, z0)
   Left_arm: (x1, y1, z1)
   Right_arm: (x2, y2, z2)
   Left_knee: (x3, y3, z3)
   .
   .
}
```

Host, Port 5052

NetworkManager (C# script in Unity)



Network Manager

```
public class NetworkManager : MonoBehaviour
{


    private UdpClient client;

    private Thread receiveThread;

    private string lastMessage = "";


    public int port = 5052;

    public PosePacket latestPose;


    // Start is called once before the first execution of Update after the MonoBehaviour is created

    void Start()
    {
        client = new UdpClient(port);

        receiveThread = new Thread(ReceiveData);
        receiveThread.IsBackground = true;
        receiveThread.Start();

        Debug.Log("NetworkManager started on port " + port);
    }


    void ReceiveData()
    {
        IPEndPoint anyIP = new IPEndPoint(IPAddress.Any, port);

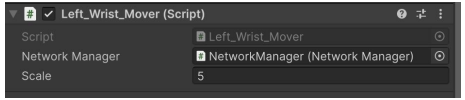
        while (true)
        {
            byte[] data = client.Receive(ref anyIP);
            string text = Encoding.UTF8.GetString(data);

            try
            {
                latestPose = JsonUtility.FromJson<PosePacket>(text);
            }
            catch { }
        }

    }
```

C# Scripts contd...

Looks for Network Manager Class, which I attached in Unity:



Left_Wrist_Mover

Reference variable to a NetworkManager object

```
using UnityEngine;

public class Left_Wrist_Mover : MonoBehaviour
{
    public NetworkManager networkManager;
    public float scale = 5f;
    // Start is called once before the first execution of Update after the MonoBehaviour is created
    void Start()
    {

    }

    // Update is called once per frame
    void Update()
    {
        if (networkManager == null)
            return;

        if (networkManager.latestPose == null)
            return;

        if (networkManager.latestPose.left_wrist == null)
            return;

        // negated x and z coordinate to make the ball appear as mirror image
        Vector3 target = new Vector3(
            -networkManager.latestPose.left_wrist.x * scale,
            networkManager.latestPose.left_wrist.y * scale,
            -networkManager.latestPose.left_wrist.z * scale
        );

        /* automatically transform the Ball that this BallMover script is attached to based
            on the vector given to it in the "target"
            It is actually inherited by the parent classes of MonoBehavior
        */
        transform.position = target;
    }
}
```

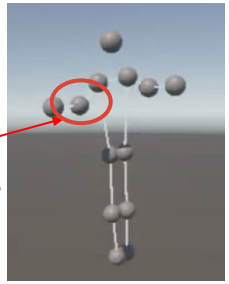
NetworkManager (C# script in Unity)

I want 'left elbow' coordinate

(x1, y1, z1)

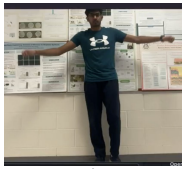
C# script of **left arm** object in Unity

Transform Left arm to (x1, y1, z1)



Calls latestPose method of the networkManager object

Methodology Overview



```
Coordinates = {
   Head: (0, 5, 2)
   Left_Arm: (5, 4, 2)
   Right_Arm: (6, 3, 2)
   ...
}
```

Host, Port 5052



From steps 1 and 2

From step 3

NetworkManager (C# script in Unity)

I want 'head' coordinate

(x0, y0, z0)

C# script of head object in Unity

Transform Head to (x0, y0, z0)

I want 'left arm' coordinate

(x1, y1, z1)

C# script of left arm object in Unity

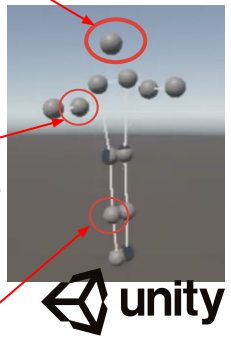
Transform Left arm to (x1, y1, z1)

I want 'left knee' coordinate

(x2, y2, z2)

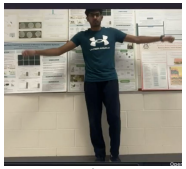
C# script of right arm object in Unity

Transform Left Knee to (x2, y2, z2)



- (Each Major body part has a C# script)

Step 3 In Context of Steps 1 and 2



```
Coordinates = {
   Head: (0, 5, 2)
   Left_Arm: (5, 4, 2)
   Right_Arm: (6, 3, 2)
   ...
}
```

Host, Port 5052



Steps 1 and 2

Step 3

NetworkManager (C# script in Unity)

I want 'head' coordinate

(x0, y0, z0)

C# script of head object in Unity

Transform Head to (x0, y0, z0)

I want 'left arm' coordinate

(x1, y1, z1)

C# script of left arm object in Unity

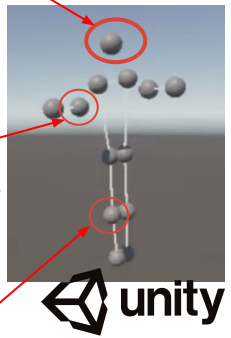
Transform Left arm to (x1, y1, z1)

I want 'left knee' coordinate

(x2, y2, z2)

C# script of right arm object in Unity

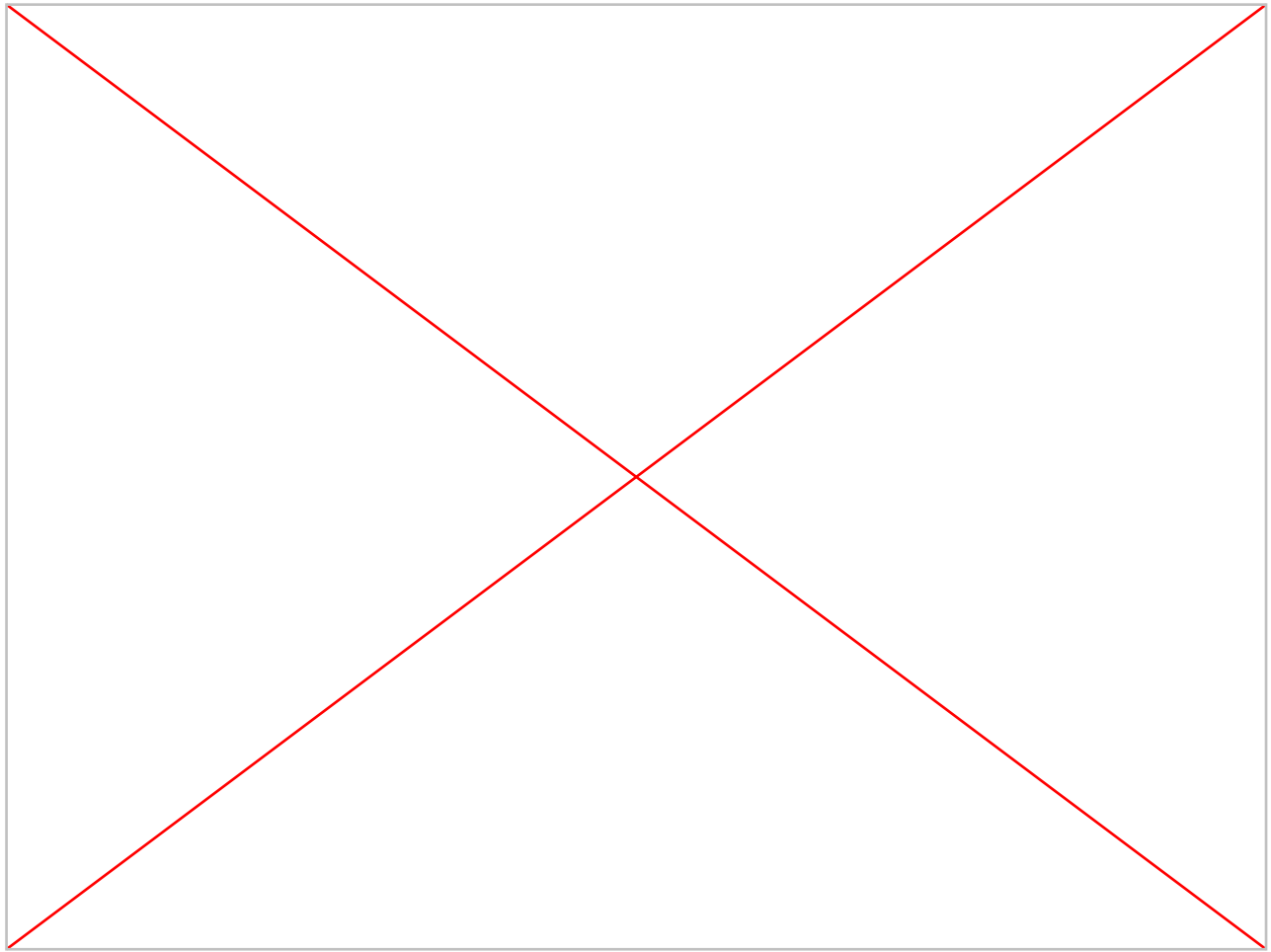
Transform Left Knee to (x2, y2, z2)



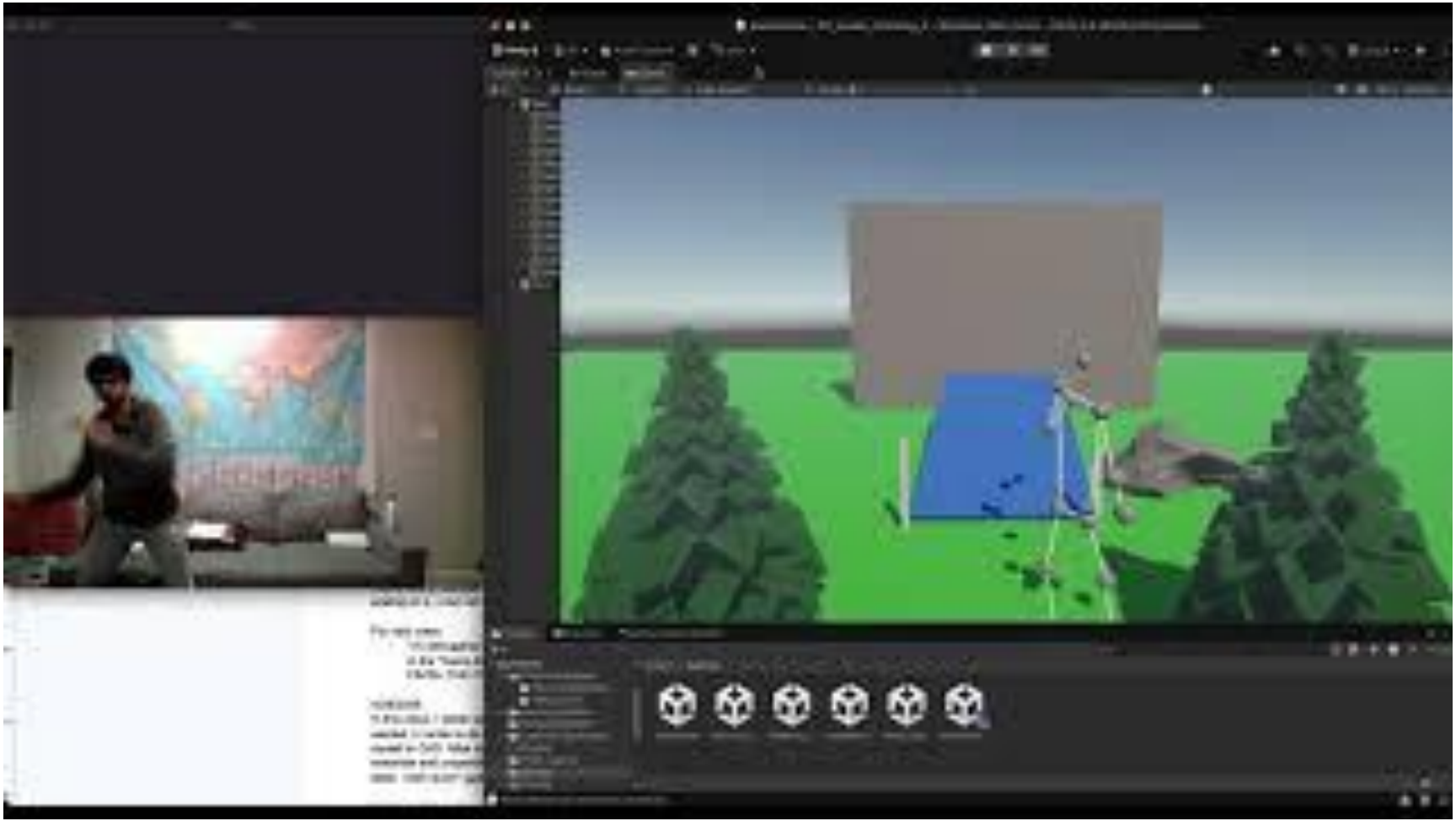
- (Each Major body part has a C# script)

RESULTS...

A working avatar



Avatar in a Table Tennis scene (Depth to be perfected next)!



CONCLUSION AND FUTURE WORK

Next Goals:

- Fixing depth perception
- Reducing Jitter
- Publicizing a Computer Vision Framework

Future Ideas:

- Surround Sound: Provide realistic sound effects
- Online & Local Multiplayer
- Custom Avatars, paddles, environments, Tables, gamerooms, etc.

REFERENCES

Figures:

- 1.) Nex Playground: https://thetoyinsider.com/nex-playground-review/
- 2.) Just Nex console: https://www.bestbuy.com/product/playground-the-active-play-system-for-kids-families-indoor-physical-play-meets-interactive-family-fun-other/J3Q5RY2VVT/sku/6593566?extStoreId=1022
- 3.) Novelty image: https://xplora.bg/en/content-marketing-en/what-is-novelty/
- 4.) Health metrics image generated by Google Gemini AI

REFERENCES

Figures (continued…):

- 5.) MediaPipe: https://www.assemblyai.com/blog/mediapipe-for-dummies
- 6.) OpenPose: https://blog.roboflow.com/what-is-openpose/
- 7.) Google's official MediaPipe documentation: https://ai.google.dev/edge/mediapipe/solutions/vision/pose_landmarker

Sources:

- 1.) https://youtu.be/nCIKEeTXdz8?si=_yvGiznrb7hIv-7x

THANK YOU!

Questions? Ideas? Suggestions?

